PortaMail

Autonomous Mail Delivery Robot Solution

Product Development Plan

Document Number	2
Revision	A
Date	1/29/2026

Approvals	Name	Date
Project Manager	Sebastian Mejias	1/29/2026
Team Member	Henry Lovelace	1/29/2026
Team Member	Pranay Dhoopar	1/29/2026
Team Member	Siddharth Ganti	1/29/2026
Team Member	David Ezuma	1/29/2026

Revision History

DCN	REV	DESCRIPTION	DATE
	A	Initial Release	

1.0 Purpose

The purpose of PortaMail is to solve the problem of labor-intensive and inefficient internal logistics in large facilities such as hospitals and corporate campuses. Current manual delivery methods face labor shortages and high costs. To address this, the project focuses on delivering a functional hardware prototype capable of navigating indoor hallways autonomously to transport mail and parcels securely.

2.0 Scope

The project scope is strictly limited to design, fabrication, and validation of the following core functions:

- Autonomous Navigation: Navigation logic (SLAM/path planning) to traverse from a central mail room to specific rooms on a single floor.
    - The area that the robot will be operating in will be pre-mapped.
- Payload handling: A custom 3D-printed bucket system with two distinct slots capable of carrying up to 15 lbs.
- Safety Systems: Emergency killswitch
- User interface: a digital screen at angle for users to schedule which room to deliver to
- Power Management: Onboard battery management to sustain operations for at least 2 hours.

3.0 Acronyms

DHF Design History File

ROS Robot Operating System

PDP Product Development Plan

PRS Product Requirement Specification

BOM Bill of Materials

CBD Conceptual Block Diagram

FBD Functional Block Diagram

WBS Work Breakdown Structure

NLD Network Logic Diagram

ConOps Concept of Operations

UI User Interface

SLAM Simultaneous Localization and Mapping

IMU Inertial Measurement Unit

LiDAR Light Detection and Ranging

PWM Pulse Width Modulation

UART Universal Asynchronous Receiver-Transmitter

I2C Inter-Integrated Circuit

PID Proportional-Integral-Derivative

MCU Microcontroller Unit

HLC High-Level Control

LLC Low-Level Control

PDR Preliminary Design Review

FTP Formal Technical Proposal

4.0 Project Management

4.1 Personnel

Project Manager - Sebastian Mejias

Mechanical Engineer - Pranay Dhoopar

Hardware Engineer - Siddharth Ganti

Software Engineer - Henry Lovelace

4.1.1 Responsibilities and Authorities

Project Manager - Manages the team, sets deadlines and is in charge of PCB and electrical design.

Mechanical Engineer - CAD Design of full robot assembly, and collaborate to set up the simulation.

Hardware Engineer - Will perform assembly/build the robot, work with software/mechanical engineer to troubleshoot issues.

Software Engineer - In charge of writing the code for the high level systems such as SLAM, LiDAR and ROS.

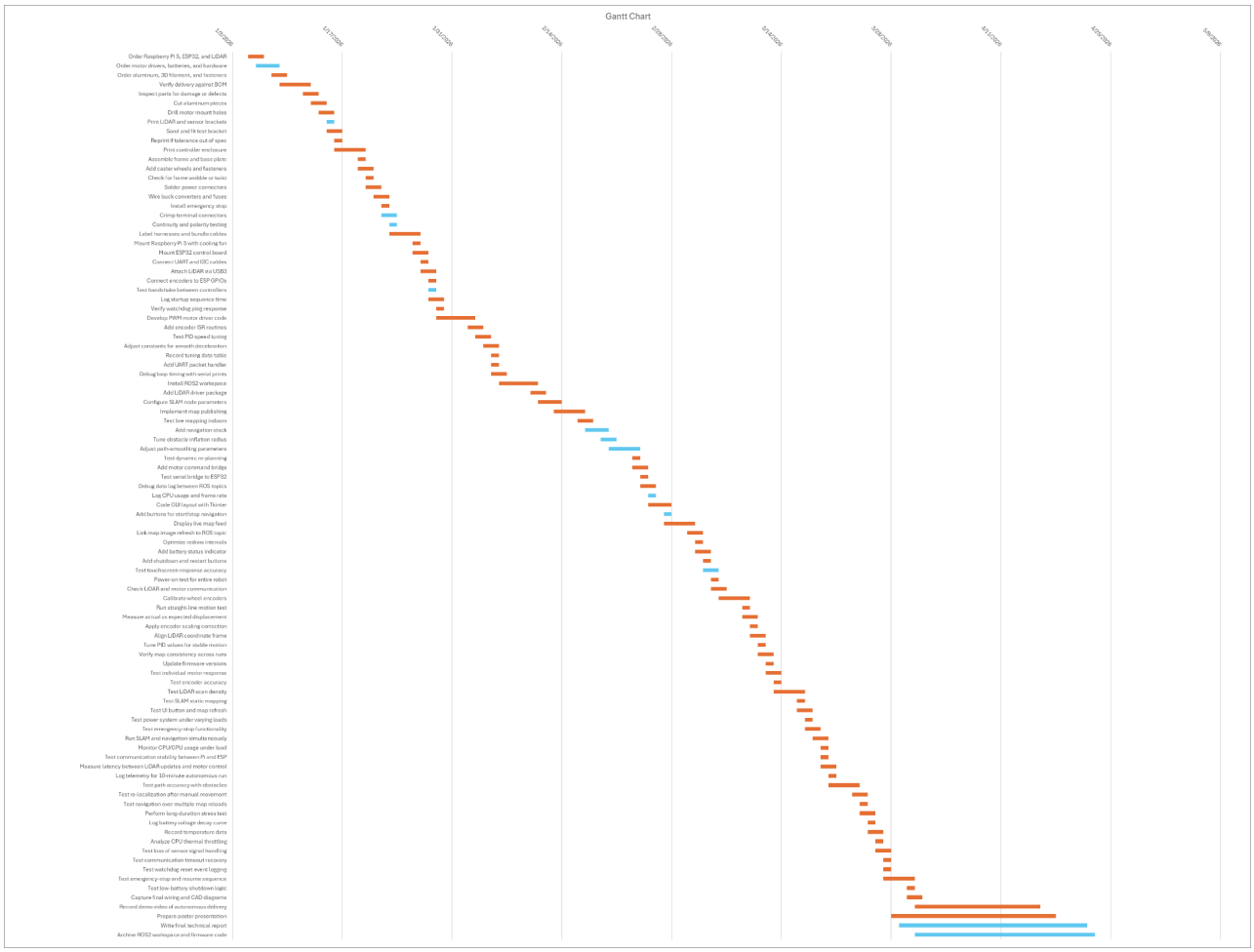
Software Engineer - In charge of writing code for the lower level systems such as motor drivers and IMU.

4.1.2 Personnel Assignment

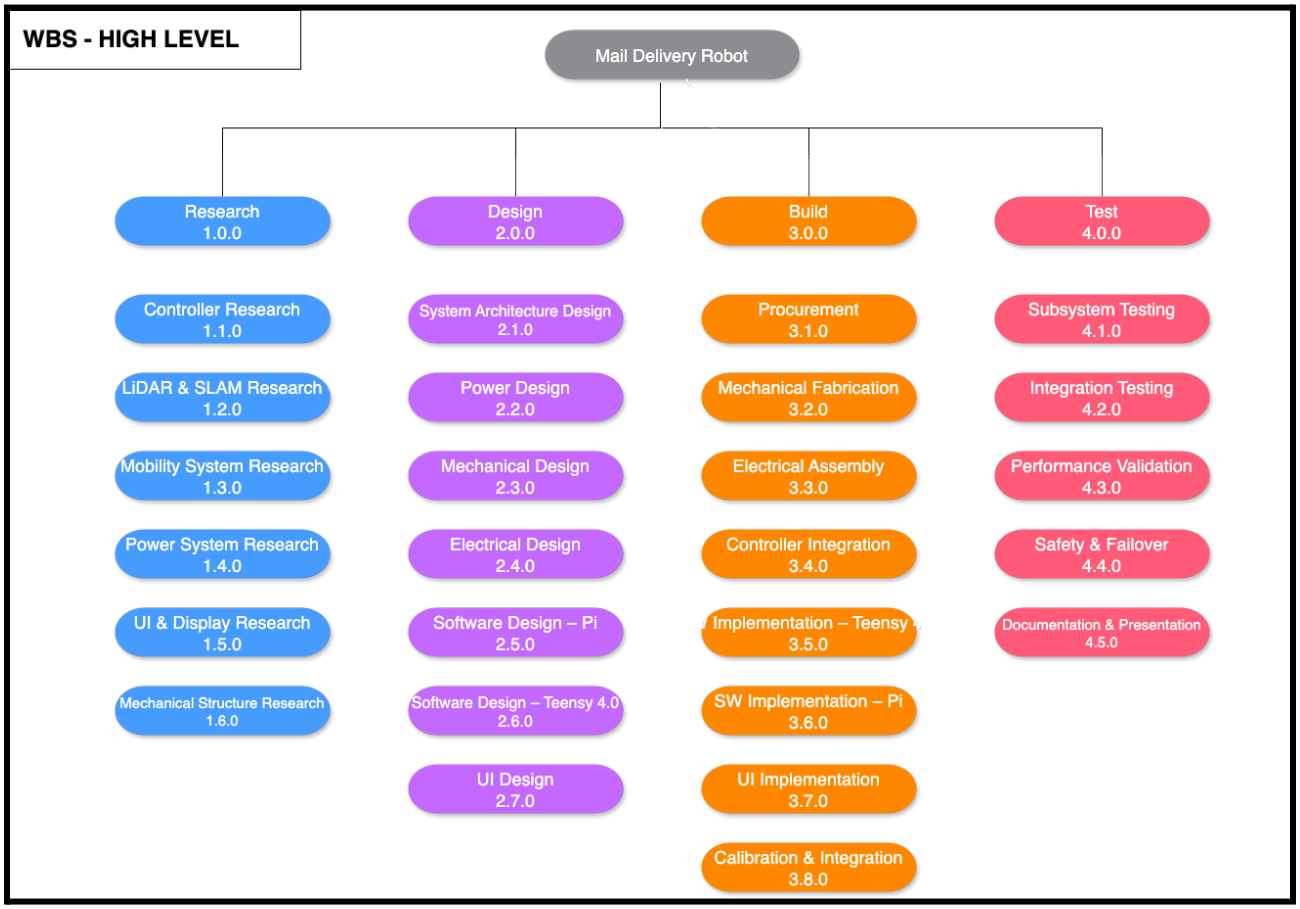
Task	*Personnel*
Design the PCB that will be used to connect the electrical components	Sebastian Mejias
Design and model the robot in CAD and prepare parts for 3D printing	Pranay Dhoopar
Bridge the software and mechanical sides and ensure that everything integrated properly	Siddharth Ganti
Write, debug and implement code for the teensy that will control the low level devices	Henry Lovelace

Write, debug and implement code for the raspberry pi 5 that will control the LiDAR. Also create a basic GUI for the Pi touchscreen	David Ezuma

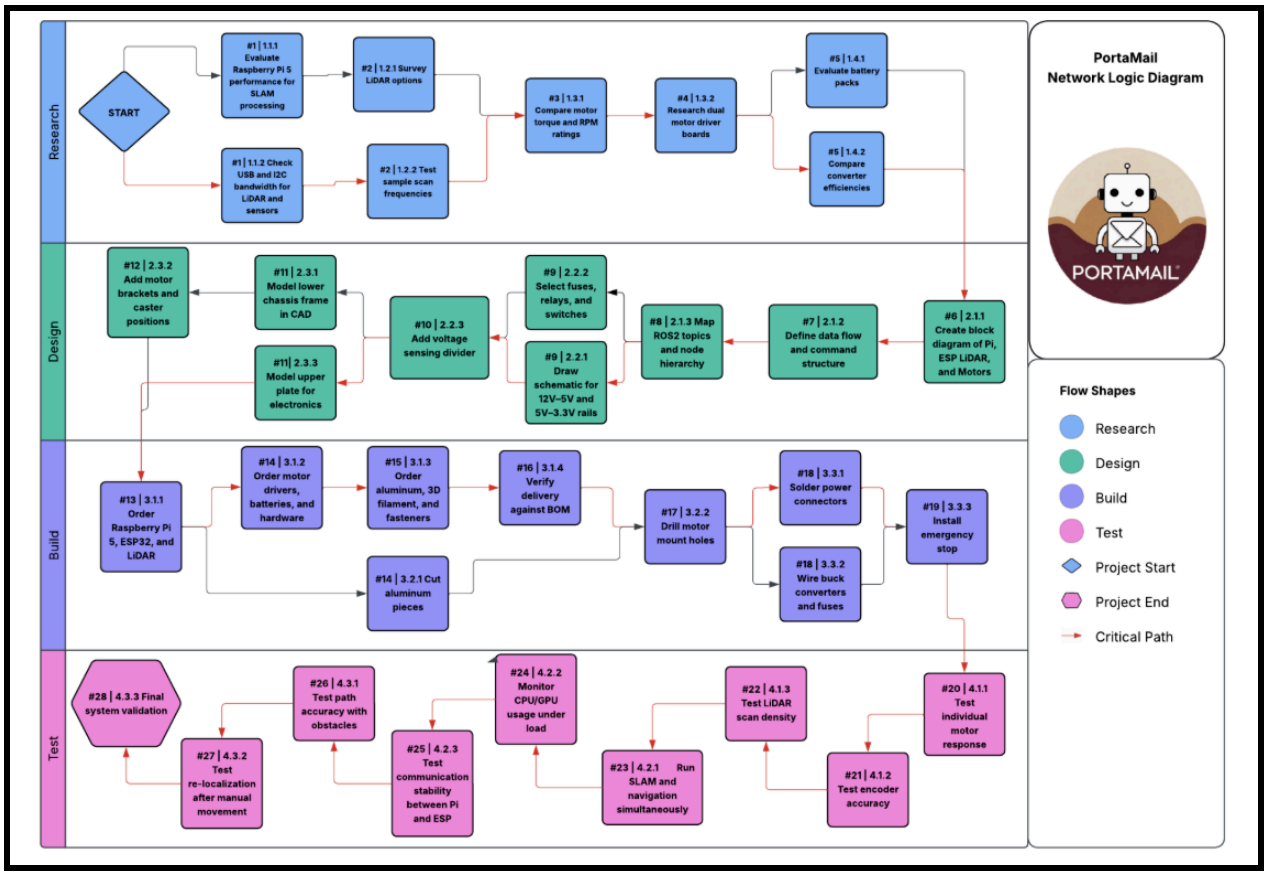
5.0 Gantt Chart Rev 01



6.0 Work Breakdown Structure



7.0 Network Diagram


PortaMail
Network Logic Diagram
PORTAMAIL
Research
Design
Build
Test
START
#1 | 1.1.1 Evaluate Raspberry Pi 5 performance for SLAM processing
#1 | 1.1.2 Check USB and I2C bandwidth for LiDAR and sensors
#2 | 1.2.1 Survey LiDAR options
#2 | 1.2.2 Test sample scan frequencies
#3 | 1.3.1 Compare motor torque and RPM ratings
#4 | 1.3.2 Research dual motor driver boards
#5 | 1.4.1 Evaluate battery packs
#5 | 1.4.2 Compare converter efficiencies
#6 | 2.1.1 Create block diagram of Pi, ESP LiDAR, and Motors
#7 | 2.1.2 Define data flow and command structure
#8 | 2.1.3 Map ROS2 topics and node hierarchy
#9 | 2.2.1 Draw schematic for 12V–5V and 5V–3.3V rails
#9 | 2.2.2 Select fuses, relays, and switches
#10 | 2.2.3 Add voltage sensing divider
#11 | 2.3.1 Model lower chassis frame in CAD
#11| 2.3.3 Model upper plate for electronics
#12 | 2.3.2 Add motor brackets and caster positions
#13 | 3.1.1 Order Raspberry Pi 5, ESP32, and LiDAR
#14 | 3.1.2 Order motor drivers, batteries, and hardware
#15 | 3.1.3 Order aluminum, 3D filament, and fasteners
#16 | 3.1.4 Verify delivery against BOM
#14 | 3.2.1 Cut aluminum pieces
#17 | 3.2.2 Drill motor mount holes
#18 | 3.3.1 Solder power connectors
#18 | 3.3.2 Wire buck converters and fuses
#19 | 3.3.3 Install emergency stop
#20 | 4.1.1 Test individual motor response
#21 | 4.1.2 Test encoder accuracy
#22 | 4.1.3 Test LiDAR scan density
#23 | 4.2.1 Run SLAM and navigation simultaneously
#24 | 4.2.2 Monitor CPU/GPU usage under load
#25 | 4.2.3 Test communication stability between Pi and ESP
#26 | 4.3.1 Test path accuracy with obstacles
#27 | 4.3.2 Test re-localization after manual movement
#28 | 4.3.3 Final system validation
Flow Shapes
Research
Design
Build
Test
Project Start
Project End
Critical Path